

Football Club & Player Management

Quản lý câu lạc bộ và cầu thủ giải European Elite League — Java Console & OOP

Mã đề: J1.L.P0036 • LOC: 300 • Học kỳ SU26

GVHD: Lê Võ Minh Thư

BÀI GIẢNG & TIÊU CHÍ CHẤM

RBL Insight Framework · Computational Thinking · AI Reflection (30%)

Mục lục

1 Khung RBL Insight, Tư duy Tính toán và Định hướng học tập	2
1.1 Vì sao LAB211 không chỉ là "viết cho chạy"	2
1.2 Research-Based Learning (RBL) là gì	2
1.3 Computational Thinking: bốn trụ cột	2
1.4 Cấu trúc báo cáo CT bắt buộc	3
1.5 Chuẩn đầu ra & trực năng lực của tài liệu	3
1.6 Hướng dẫn viết báo cáo & trình bày	4
1.7 Tư duy về độ phức tạp & trường hợp biên	4
2 Phân tích đề bài Football Club & Player	5
2.1 Bối cảnh	5
2.2 Hai thực thể: quan hệ 1-nhiều	5
2.3 Định dạng dữ liệu	5
2.4 Ràng buộc (Validation Rules)	6
2.5 Mười bốn chức năng	6
3 Áp dụng Computational Thinking	7
3.1 Decomposition — Phân rã	7
3.1.1 Theo dữ liệu	7
3.1.2 Theo chức năng	7
3.1.3 Theo class (OOP View)	7
3.2 Pattern Recognition — Nhận diện mẫu	7
3.3 Abstraction — Trừu tượng hoá	7
3.4 Algorithm Design — Thiết kế thuật toán	7
3.4.1 Workflow: Add a new player (Function 8)	7
3.4.2 Workflow: Sort players (Function 6)	8
4 Thiết kế hướng đối tượng (OOP)	9
4.1 Use Case Diagram	9
4.2 Class Diagram	9
4.3 Sequence Diagram — Add Player	10
4.4 Bốn nguyên lý OOP qua code	10
4.4.1 Encapsulation — lớp Club	10
4.4.2 Abstraction + Inheritance: Person → Player	10
4.4.3 Polymorphism: Comparator chaining (Function 6)	11
4.4.4 Interface + REGEX: Validatable	11
4.5 Cấu trúc package	12
5 Triển khai chức năng & code mẫu	13
5.1 Menu chính (14 chức năng)	13
5.2 Thêm club (Function 2) & tìm theo ID (Function 3)	13
5.3 Lọc club theo budget (Function 5) & tìm cầu thủ theo tên (Function 7)	13
5.4 Cập nhật player (Function 10) — giữ field nếu để trống	14
5.5 Đọc/ghi tệp văn bản (Functions 12, 13)	14
5.6 Mẫu kết quả (Sample Output)	15
5.7 Liệt kê & xóa cầu thủ, lọc theo vị trí, thoát	15
5.8 Phiên chạy mẫu đầy đủ	16
5.9 Flowchart bổ sung: Load chặt (Function 13)	17

6 Tiêu chí chấm (Rubric)	18
6.1 Cấu trúc tổng thể (thang 100, quy đổi 300 LOC)	18
6.2 Chi tiết chức năng (15đ) — bám đúng bảng trường	18
6.3 Chi tiết CT (20đ) & OOP (25đ)	18
6.4 Worked Example & lỗi mất điểm	19
6.5 AI Reflection (30đ)	19
7 AI Audit Log & AI Reflection — Cách làm và cách chấm	20
7.1 AI Audit Log là gì và <i>không</i> là gì	20
7.2 Core Prompt vs. Auxiliary Prompt	20
7.3 Yêu cầu số lượng (bài Assignment LAB211)	20
7.4 Cấu trúc một entry: 7 phần	20
7.5 Phát hiện Hallucination (bắt buộc)	21
7.6 Tiêu chí chấm AI Reflection (30%)	21
7.7 Dấu hiệu nhận biết khi chấm	22
7.8 Quy định cho sinh viên	22
7.9 AI Literacy — viết prompt hiệu quả	22
7.10 Phương pháp Oral Vivas (vấn đáp)	23
7.11 Bảng phân bố entries theo thành phần CT	23
8 Ví dụ AI Audit Log cho Football	24
8.1 Entry ĐẠT — Verification (regex CL-xxxx)	24
8.2 Entry ĐẠT — Decision (Comparator chaining)	24
8.3 Entry phát hiện HALLUCINATION (số áo & vị trí)	24
8.4 Entry KÉM (để đối chiếu)	24
8.5 Ngân hàng câu hỏi vấn đáp (Football)	25
8.6 Chỉ mục Audit Log mẫu (20 core prompts)	25
Phụ lục — Checklist tự chấm	25

1

Khung RBL Insight, Tư duy Tính toán và Định hướng học tập

1.1 Vì sao LAB211 không chỉ là "viết cho chạy"

Trong môn **LAB211**, một chương trình chạy đúng yêu cầu chỉ là điều kiện *cần*. Điều kiện *đủ* — và là phần tạo ra giá trị học tập thực sự — nằm ở chỗ sinh viên **hiểu vì sao** mình thiết kế như vậy, **chứng minh được** quá trình tư duy, và **kiểm soát được** mọi dòng code (kể cả code do AI gợi ý). Đây chính là tinh thần của **RBL Insight Framework**.

Tuyên ngôn cốt lõi

AI không thay thế sinh viên. AI là trợ lý hỗ trợ tư duy, phân tích, gỡ lỗi và thiết kế để quá trình học tập của sinh viên được thuận lợi. Giá trị được chấm điểm là phần **con người thêm vào** mà AI không tự làm được — gọi là *Human Delta*.

Ba năng lực mà tài liệu này rèn cho người học, đặt cạnh nhau và bổ trợ lẫn nhau:



Từ phân tích vấn đề → thiết kế hướng đối tượng
→ minh bạch và phản biện quá trình dùng AI.

Hình 1: Ba năng lực cốt lõi: Tư duy tính toán, OOP và AI Reflection

1.2 Research-Based Learning (RBL) là gì

RBL — Research-Based Learning là cách học lấy *quá trình tìm tòi, đặt câu hỏi, thử nghiệm và phản biện* làm trung tâm, thay vì chỉ tiếp nhận lời giải có sẵn. Khi kết hợp với **PBL (Project-Based Learning)**, sinh viên học thông qua một dự án thực tế và phải tự trả lời các câu hỏi nghiên cứu nhỏ:

- **Đặt vấn đề:** Yêu cầu thực sự là gì? Ràng buộc nào là cốt lõi?
- **Giả thuyết & phương án:** Có những cách tiếp cận nào? Ưu/nhược điểm?
- **Thử nghiệm & đo lường:** Phương án nào tốt hơn, dựa trên bằng chứng gì?
- **Kết luận & sở hữu quyết định:** Tôi chọn gì, vì sao, và tôi chịu trách nhiệm về lựa chọn đó.

RBL Insight gắn với AI thế nào?

"Insight" ở đây là *bằng chứng tư duy* mà sinh viên để lại trong suốt quá trình làm dự án. Khi sinh viên dùng AI, bằng chứng đó được ghi lại có chọn lọc trong **AI Audit Log** — không phải để "khoe đã hỏi gì" mà để chứng minh *đã suy nghĩ, đã phản biện, đã quyết định*.

1.3 Computational Thinking: bốn trụ cột

Computational Thinking (CT) là phương pháp sư phạm rèn tư duy giải quyết vấn đề theo cách tiếp cận của khoa học máy tính. CT gồm bốn thành phần, và toàn bộ phần "Bài giảng"

của tài liệu này được tổ chức quanh bốn trụ cột đó.



Hình 2: Computational Thinking: bốn trụ cột

- **Decomposition — Phân rã.** Chia bài toán theo *dữ liệu*, theo *chức năng* và theo *lớp (OOP view)*. Mỗi phần nhỏ có thể giải độc lập rồi ghép lại.
- **Pattern Recognition — Nhận diện mẫu.** Tìm điểm chung: các quy luật *validate* (dùng Regular Expression), các thao tác *CRUD* lặp lại giữa các thực thể → tạo template/hàm dùng chung.
- **Abstraction — Trừu tượng hoá.** Với mỗi đối tượng, xác định thuộc tính và hành vi cốt lõi; với mỗi chức năng, chỉ quan tâm *input* → *xử lý* → *output*, bỏ qua chi tiết không cần thiết.
- **Algorithm Design — Thiết kế thuật toán.** Mô tả *Operation Workflow* bằng pseudocode/flowchart *trước khi viết code Java*.

Quy tắc vàng khi áp dụng CT

Luôn đi theo trình tự: **hiểu yêu cầu** → **phân rã** → **nhận diện mẫu** → **trừu tượng hoá thành lớp** → **viết pseudocode** → **rồi mới code**. "Code trước, nghĩ sau" là nguyên nhân số một của lỗi logic và của những bài làm không giải thích được.

1.4 Cấu trúc báo cáo CT bắt buộc

Phần trình bày/báo cáo của sinh viên cần bám theo bộ khung sau (giảng viên dùng chính khung này để đối chiếu khi chấm CT):

1. Introduction — giới thiệu bài toán và mục tiêu.
2. Problem Analysis — phân tích yêu cầu, dữ liệu, ràng buộc.
3. Computational Thinking: 3.1 Decomposition, 3.2 Pattern Recognition, 3.3 Abstraction, 3.4 Algorithm Design.
4. OOP Design (UML) — use case, class diagram (và sequence nếu có).
5. Flowchart — sơ đồ luồng cho các chức năng chính.
6. Conclusion — tổng kết, hạn chế, hướng mở rộng.

1.5 Chuẩn đầu ra & trực năng lực của tài liệu

Tài liệu được thiết kế bám sát các chuẩn đầu ra của học phần LAB211: vận dụng tư duy tính toán để phân tích bài toán, áp dụng lập trình hướng đối tượng để thiết kế giải pháp, và sử dụng công cụ AI một cách có kiểm chứng. Bốn trực năng lực được rèn luyện và đánh giá xuyên suốt như sau:

Trực năng lực → phần tài liệu tương ứng	
Tư duy CT	Hiểu & áp dụng 4 thành phần vào dự án cụ thể (Chương CT).
OOP vững	Xác định class, đóng gói, kế thừa, đa hình (Chương OOP).
Chất lượng dùng AI	Kiểm chứng, hiểu trước khi dùng, chỉnh sửa output (AI Audit Log).
Phản biện (AI Reflection)	Human Delta, phát hiện hallucination, tự đánh giá lại.
AI Literacy	Nguyên tắc dùng GenAI, viết prompt hiệu quả, hiểu rủi ro & đạo đức.

20đ

Computational Thinking

25đ

OOP Design & Quality

30đ

AI Reflection

25đ

Chức năng + Code style

(Thang quy đổi 100 điểm — chi tiết ở chương Tiêu chí chấm.)

1.6 Hướng dẫn viết báo cáo & trình bày

Báo cáo tốt là báo cáo *chứng minh tư duy*, không phải liệt kê code. Bộ khung mẫu để sinh viên triển khai sáu mục đã nêu:

Mục	Nội dung cần có (gợi ý 0.5-1 trang/mục)
1. Introduction	Bài toán là gì, ai dùng, mục tiêu, phạm vi.
2. Problem Analysis	Dữ liệu vào/ra, ràng buộc, các trường hợp biên (edge cases).
3. Computational Thinking	4 tiểu mục: sơ đồ phân rã, bảng mẫu lặp, bảng thuộc tính trừu tượng, pseudo-code/flowchart.
4. OOP Design (UML)	Use case + class diagram + (tuỳ chọn) sequence; giải thích quan hệ.
5. Flowchart	Sơ đồ luồng cho 2-3 chức năng phức tạp nhất.
6. Conclusion	Đã đạt gì, hạn chế, hướng mở rộng, bài học rút ra.

Chuẩn bị trình bày (Oral / Demo)

Chuẩn bị một *kịch bản demo 5 phút*: (1) chạy menu, (2) thêm một bản ghi hợp lệ và một bản ghi bị chặn (để chứng minh validate), (3) một chức năng truy vấn, (4) thống kê, (5) lưu & thoát có nhắc lưu. Vừa thao tác vừa giải thích vì *sao* code xử lý như vậy.

1.7 Tư duy về độ phức tạp & trường hợp biên

Một phần quan trọng của CT là *lường trước* cái gì có thể sai. Trước khi code mỗi chức năng, hãy liệt kê edge cases:

Nhóm	Trường hợp biên cần xử lý
Danh sách rỗng	Hiển thị/tìm/thống kê khi chưa có dữ liệu → thông báo phù hợp, không crash.
Trùng khoá	Thêm bản ghi có ID đã tồn tại → chặn.
Khoá ngoại sai	Tham chiếu tới mã không tồn tại → báo lỗi, yêu cầu nhập lại.
Nhập sai kiểu	Nhập chữ khi cần số, vượt khoảng cho phép → vòng lặp nhập lại.
Hủy thao tác	Người dùng chọn "N" khi xác nhận xoá → giữ nguyên dữ liệu.
Thoát chưa lưu	Có thay đổi mà chưa lưu → nhắc lưu (cờ dirty).

Liên hệ độ phức tạp

Khi danh sách lớn, lựa chọn cấu trúc dữ liệu ảnh hưởng hiệu năng: tra cứu theo khoá nên dùng `HashMap` ($O(1)$) thay vì duyệt `List` ($O(n)$); sắp xếp đa tiêu chí nên dùng `Comparator chaining`. Những lựa chọn này chính là *core prompts* đáng ghi vào Audit Log.

2

Phân tích đề bài Football Club & Player

2.1 Bối cảnh

Giải đấu *European Elite League (EEL)* cần số hoá việc quản lý các câu lạc bộ (**Club**) và cầu thủ (**Player**). Công ty phần mềm FTC — nơi sinh viên đóng vai lập trình viên Java — xây dựng một ứng dụng console. Dữ liệu lưu trong hai tệp văn bản, *tự nạp khi khởi động*, và có chức năng nạp lại theo yêu cầu:

- `clubs.txt` — thông tin câu lạc bộ.
- `players.txt` — thông tin cầu thủ.

Yêu cầu nền tảng

Phân tích & thiết kế theo **OOP** (abstraction, encapsulation, inheritance, polymorphism); trình bày báo cáo theo **Computational Thinking**; validation phải *chặt chẽ*.

2.2 Hai thực thể: quan hệ 1-nhiều



Hình 3: Hai thực thể: quan hệ 1-nhiều

Một **Club** có nhiều **Player**; **Player** phụ thuộc **Club** (`clubID` phải tồn tại). Quan hệ này chi phối các ràng buộc (khóa ngoại) và các chức năng (thêm cầu thủ phải kiểm tra club tồn tại; số áo duy nhất *trong cùng club*).

2.3 Định dạng dữ liệu

Listing 2.1. `clubs.txt`: <Club ID, club name, sponsor brand, budget> (triệu EUR)

```
CL-0001, FC Barcelona, Nike, 820
CL-0002, Real Madrid Galacticos, Adidas, 910
CL-0012, Chelsea FC XLine, Nike, 780
```

Listing 2.2. `players.txt`: <Player ID, club ID, player name, position, shirt number>

```
P0001, CL-0001, Lionel Messi, Forward, 10
P0002, CL-0001, Sergio Busquets, Midfielder, 5
P0013, CL-0012, Raheem Sterling, Winger, 7
```

2.4 Ràng buộc (Validation Rules)

Trường	Quy tắc	Mẫu / thông báo lỗi
Club ID	Duy nhất; định dạng CL-xxxx.	CL-0001; "This club ID already exists!"
Club name	Không rỗng.	FC Barcelona
Sponsor	Không rỗng.	Nike
Budget	Số thực dương (triệu EUR).	820
Player ID	Duy nhất; định dạng Pxxxx.	P0001; "This player ID already exists!"
Club ID (player)	Phải tồn tại trong danh sách club.	"This club does not exist!"
Position	$\in \{\text{Goalkeeper, Defender, Midfielder, Forward, Winger}\}$, không phân biệt hoa thường.	Forward
Shirt number	Số nguyên 1-99, duy nhất trong cùng club.	"This shirt number already exists in this club!"

2.5 Mười bốn chức năng

#	Chức năng	Mô tả & thông báo
1	List all clubs	Bảng: Club ID, Club Name, Sponsor, Budget.
2	Add a new club	Validate; chặn trùng ID; báo thành công/thất bại.
3	Search club by ID	Không có $\rightarrow$ "This club does not exist!".
4	Update club by ID	Enter rỗng giữ nguyên field; validate khi sửa.
5	List clubs with budget $\leq$ input	Nhập ngưỡng; lọc theo budget.
6	List players sorted by club name, then shirt number	Sắp tăng dần theo tên club, cùng club thì theo số áo.
7	Search players by partial name	So khớp một phần, không phân biệt hoa thường.
8	Add a new player	Validate đủ ràng buộc + số áo duy nhất trong club.
9	Remove a player by ID	Không có $\rightarrow$ "This player does not exist!".
10	Update a player by ID	Enter rỗng giữ field; kiểm tra số áo khi đổi.
11	List players by position	Lọc theo vị trí.
12	Save data to files	Ghi clubs.txt & players.txt; báo hoàn tất.
13	Load data from files	Xoá hiện tại, nạp club rồi player, validate chặt; "Load data failed!" / "successfully!".
14	Quit program	Có thay đổi $\rightarrow$ lưu trước khi thoát.

3

Áp dụng Computational Thinking

Phần này theo đúng mô hình mẫu *Apply_CT_toProject* của bộ môn (xem lý thuyết ở §1.3).

3.1 Decomposition — Phân rã

3.1.1 Theo dữ liệu

Hai thực thể **Club**, **Player** với quan hệ 1-nhiều; đọc/ghi từ `clubs.txt` và `players.txt`; lưu thay đổi khi thoát.

3.1.2 Theo chức năng



Hình 4: Theo chức năng

3.1.3 Theo class (OOP View)

Các lớp: `Club`, `Player` (model); `ClubsManager`, `PlayersManager` (business); `FileUtils`, `ValidationUtils`, `Menu` (tools/dispatcher).

3.2 Pattern Recognition — Nhận diện mẫu

- **Validate bằng Regex:** `CL-\d{4}` cho club, `P\d{4}` cho player; `isValid(value, pattern)` dùng chung.
- **CRUD template:** Add/Search/Update/Remove lặp lại trên cả Club lẫn Player → tách logic chung; riêng dữ liệu *chỉ đọc* thì chỉ cần nạp để validate.
- **Sắp xếp đa tiêu chí:** chức năng 6 dùng Comparator chaining (club name → shirt number).

3.3 Abstraction — Trừu tượng hoá

- **Club:** clubID, name, sponsor, budget (+ hành vi hiển thị, so sánh theo tên).
- **Player:** playerID, clubID, name, position, shirtNumber.
- Theo chức năng: "Add a new Club" chỉ quan tâm *validate* → *add* → *message*, không cần biết menu.

3.4 Algorithm Design — Thiết kế thuật toán

3.4.1 Workflow: Add a new player (Function 8)

Listing 3.1. Pseudocode — Add player với số áo duy nhất trong club

```

FUNCTION addPlayer():
    id = inputLoop("Player ID:", P_REGEX)
    IF players.exists(id) THEN PRINT "This player ID already exists!"; RETURN
    showClubList()
  
```

```

clubId = inputLoop("Club ID:", CL_REGEX)
IF NOT clubs.exists(clubId) THEN PRINT "This club does not exist!"; RETURN
name = inputNonEmpty("Name:")
pos = inputPosition() // case-insensitive, in enum
shirt= inputInt("Shirt (1-99):", 1, 99)
IF players.shirtTakenInClub(clubId, shirt)
    THEN PRINT "This shirt number already exists in this club!"; RETURN
players.add(new Player(id, clubId, name, pos, shirt)); markDirty()
PRINT "Added successfully"
END

```

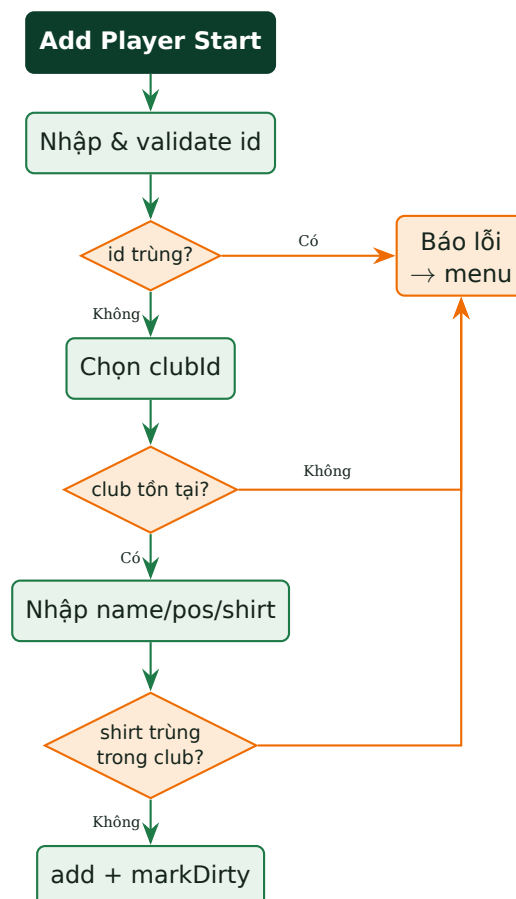
3.4.2 Workflow: Sort players (Function 6)

Listing 3.2. Pseudocode — sắp theo tên club rồi số áo

```

FUNCTION listPlayersSorted():
    copy = new list(players)
    SORT copy BY clubName(clubId) ASC, THEN shirtNumber ASC
    PRINT copy as table
END

```

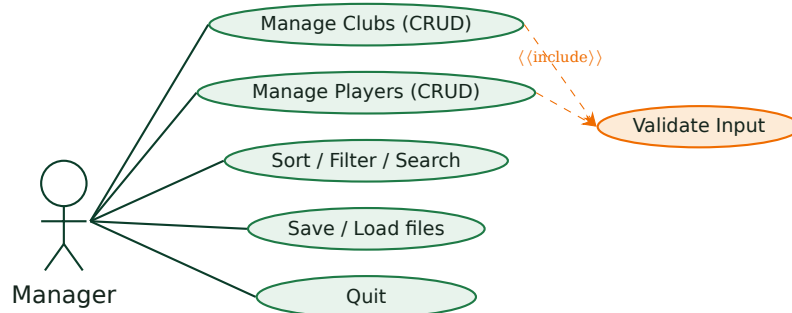


Hình 5: Workflow: Sort players (Function 6)

4

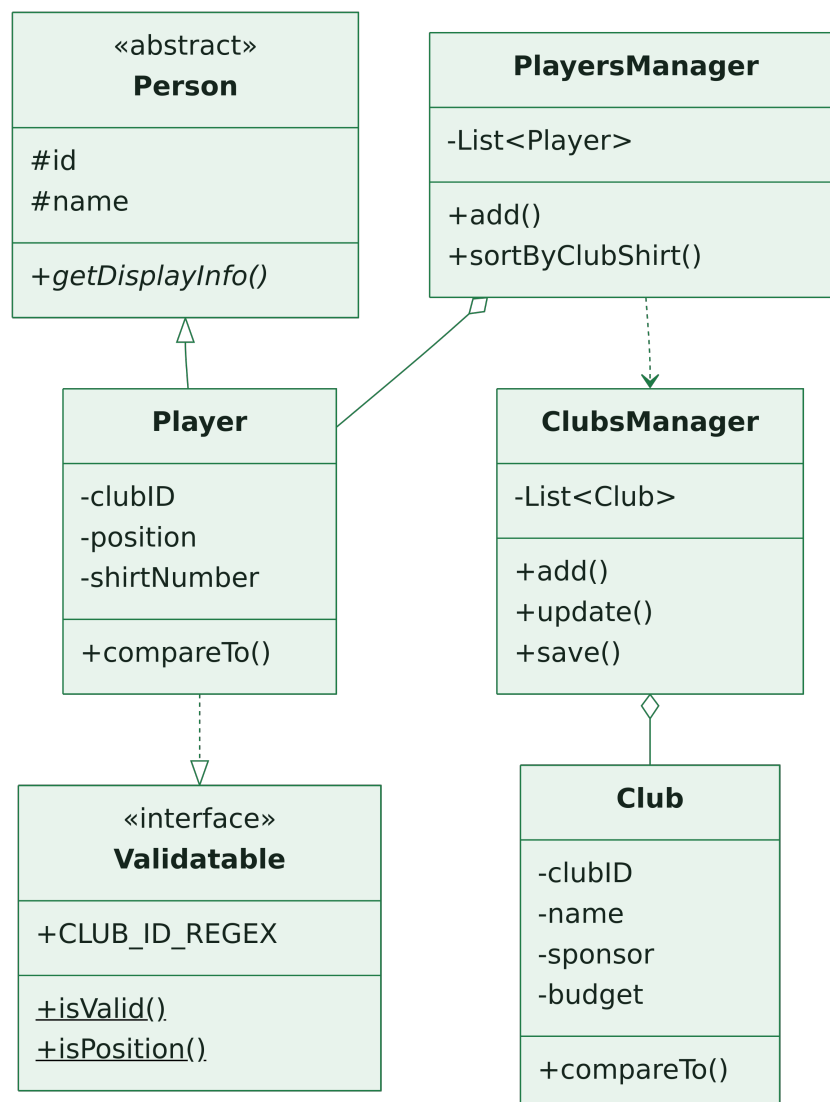
Thiết kế hướng đối tượng (OOP)

4.1 Use Case Diagram



Hình 6: Use Case Diagram

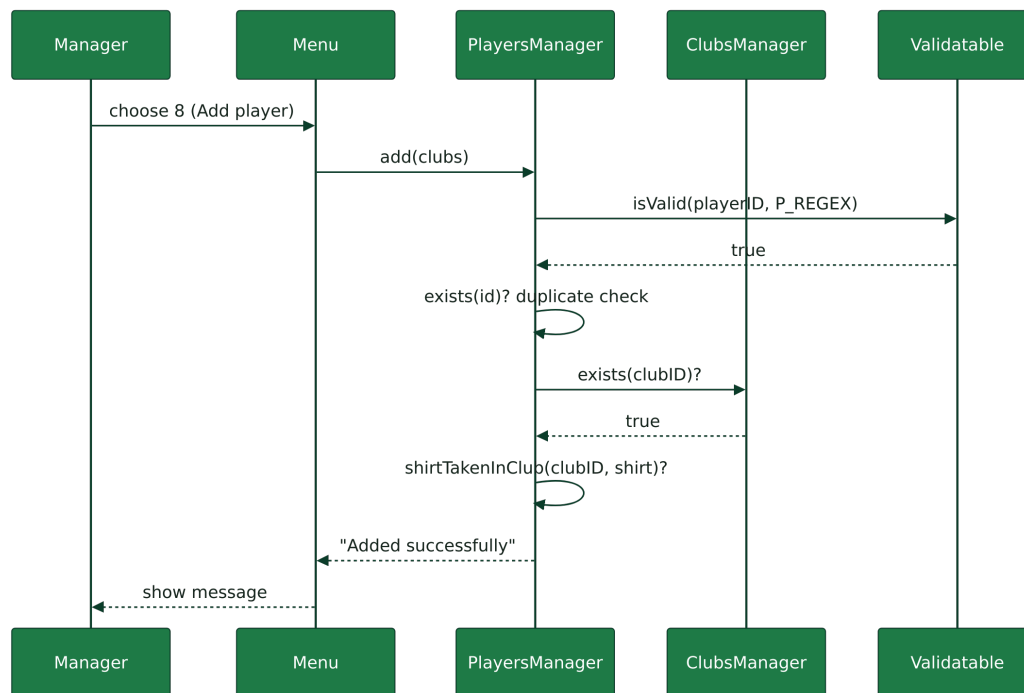
4.2 Class Diagram



Hình 7: Class Diagram

- ClubsManager *has-a* List<Club>; PlayersManager *has-a* List<Player>.
- Person (abstract) làm gốc cho Player; Validatable là interface chứa REGEX & isValid.
- Player implements Comparable<Player>; chức năng 6 dùng Comparator chaining.

4.3 Sequence Diagram — Add Player



Hình 8: Sequence Diagram — Add Player

4.4 Bốn nguyên lý OOP qua code

4.4.1 Encapsulation — lớp Club

Listing 4.1. Club: field private + getter/setter; budget kiểm tra dương

```

1 public class Club implements Serializable, Comparable<Club> {
2     private String clubID, name, sponsor;
3     private double budget;
4     public Club(String clubID, String name, String sponsor, double budget) {
5         this.clubID = clubID; this.name = name; this.sponsor = sponsor; this.budget = budget;
6     }
7     public String getClubID() { return clubID; }
8     public String getName() { return name; }
9     public double getBudget() { return budget; }
10    public void setBudget(double b) { if (b > 0) this.budget = b; }
11    @Override public int compareTo(Club o) { return this.name.compareToIgnoreCase(o.name); }
12    @Override public String toString() {
13        return String.format("%-8s| %-24s| %-8s| %,.0f", clubID, name, sponsor, budget);
14    }
15 }

```

4.4.2 Abstraction + Inheritance: Person → Player

Listing 4.2. Person abstract; Player kế thừa và override

```

1 public abstract class Person implements Serializable {
2     protected String id, name;
3     public abstract String getDisplayInfo();
4     public String getId() { return id; }
5 }
6 public class Player extends Person implements Comparable<Player> {
7     private String clubID, position; private int shirtNumber;
8     public Player(String id, String clubID, String name, String position, int shirt) {
9         this.id = id; this.clubID = clubID; this.name = name;
10        this.position = position; this.shirtNumber = shirt;
11    }
12    public String getClubID() { return clubID; }
13    public int getShirtNumber() { return shirtNumber; }
14    public String getPosition() { return position; }
15    @Override public String getDisplayInfo() { return id + " - " + name + " (#" +
16        shirtNumber + ")"; }
17    @Override public int compareTo(Player o) { return Integer.compare(shirtNumber,
18        o.shirtNumber); }
19    @Override public String toString() {
20        return String.format("%-7s| %-8s| %-18s| %-11s| %d", id, clubID, name, position,
21            shirtNumber);
22    }
23 }

```

4.4.3 Polymorphism: Comparator chaining (Function 6)

Listing 4.3. Sắp theo tên club rồi số áo, null-safe

```

1 public void listSortedByClubThenShirt(ClubsManager clubs) {
2     List<Player> copy = new ArrayList<>(list);
3     Comparator<Player> byClubName = Comparator.comparing(
4         p -> clubs.getName(p.getClubID()),
5         Comparator.nullsLast(String.CASE_INSENSITIVE_ORDER));
6     copy.sort(byClubName.thenComparingInt(Player::getShirtNumber));
7     copy.forEach(System.out::println);
8 }

```

Vì sao là "áp dụng nâng cao"?

`Comparator.comparing(...).thenComparingInt(...)` là đa hình hàm: ta truyền *hành vi so sánh* như tham số. `nullsLast` chống `NullPointerException` khi club name không tra được — một edge case quan trọng.

4.4.4 Interface + REGEX: Validatable

Listing 4.4. Một nguồn duy nhất cho quy tắc hợp lệ

```

1 public interface Validatable {
2     String CLUB_ID_REGEX = "^CL-\\d{4}$";
3     String PLAYER_ID_REGEX = "^P\\d{4}$";
4     java.util.Set<String> POSITIONS = java.util.Set.of(
5         "goalkeeper", "defender", "midfielder", "forward", "winger");
6     static boolean isValid(String v, String pattern) { return v != null &&
7         v.matches(pattern); }
8     static boolean isPosition(String v) { return v != null &&
9         POSITIONS.contains(v.toLowerCase()); }
10 }

```

4.5 Cấu trúc package

model/	Person, Club, Player
business/	ClubsManager, PlayersManager
tools/	FileUtils, ValidationUtils (impl Validatable), Inputter
dispatcher/	Menu / Main

5

Triển khai chức năng & code mẫu

5.1 Menu chính (14 chức năng)

Listing 5.1. Vòng lặp menu

```
1 do {
2     showMenu();
3     int c = Inputter.inputInt("Choose 1-14: ", 1, 14);
4     switch (c) {
5         case 1 -> clubs.listAll();
6         case 2 -> clubs.add();
7         case 3 -> clubs.searchById();
8         case 4 -> clubs.update();
9         case 5 -> clubs.listByBudget();
10        case 6 -> players.listSortedByClubThenShirt(clubs);
11        case 7 -> players.searchByPartialName();
12        case 8 -> players.add(clubs);
13        case 9 -> players.remove();
14        case 10 -> players.update(clubs);
15        case 11 -> players.listByPosition();
16        case 12 -> { clubs.save(); players.save(); }
17        case 13 -> reloadAll(clubs, players);
18        case 14 -> quit(clubs, players);
19    }
20 } while (choice != 14);
```

5.2 Thêm club (Function 2) & tìm theo ID (Function 3)

Listing 5.2. add() and searchById()

```
1 public void add() {
2     String id = Inputter.inputLoop("Club ID: ", Validatable.CLUB_ID_REGEX, "Format must be
    CL-xxxx");
3     if (findById(id) != null) { System.out.println("This club ID already exists!"); return; }
4     String name = Inputter.inputNonEmpty("Club name: ");
5     String sponsor = Inputter.inputNonEmpty("Sponsor brand: ");
6     double budget = Inputter.inputPositiveDouble("Budget (million EUR): ");
7     list.add(new Club(id, name, sponsor, budget)); dirty = true;
8     System.out.println("Club added successfully.");
9 }
10 public void searchById() {
11     String id = Inputter.inputLoop("Club ID: ", Validatable.CLUB_ID_REGEX, "Format must be
    CL-xxxx");
12     Club c = findById(id);
13     System.out.println(c == null ? "This club does not exist!" : c);
14 }
```

5.3 Lọc club theo budget (Function 5) & tìm cầu thủ theo tên (Function 7)

Listing 5.3. listByBudget() and searchByPartialName()

```
1 public void listByBudget() {
```

```

2  double max = Inputter.inputPositiveDouble("Max budget: ");
3  boolean any = false;
4  for (Club c : list) if (c.getBudget() <= max) { System.out.println(c); any = true; }
5  if (!any) System.out.println("No clubs match the budget condition.");
6  }
7  public void searchByPartialName() {
8      String kw = Inputter.inputNonEmpty("Partial player name: ").toLowerCase();
9      List<Player> found = new ArrayList<>();
10     for (Player p : list) if (p.getName().toLowerCase().contains(kw)) found.add(p);
11     if (found.isEmpty()) System.out.println("No players match the search criteria!");
12     else found.forEach(System.out::println);
13 }

```

5.4 Cập nhật player (Function 10) — giữ field nếu để trống

Listing 5.4. update() với kiểm tra số áo trùng trong club

```

1  public void update(ClubsManager clubs) {
2      String id = Inputter.inputLoop("Player ID: ", Validatable.PLAYER_ID_REGEX, "Format must
        be Pxxxx");
3      Player p = findById(id);
4      if (p == null) { System.out.println("This player does not exist!"); return; }
5
6      String name = Inputter.inputOptional("Name [" + p.getName() + "]: ");
7      if (!name.isEmpty()) p.setName(name);
8
9      String pos = Inputter.inputOptional("Position [" + p.getPosition() + "]: ");
10     if (!pos.isEmpty() && Validatable.isPosition(pos)) p.setPosition(pos);
11
12     String shirtStr = Inputter.inputOptional("Shirt [" + p.getShirtNumber() + "]: ");
13     if (!shirtStr.isEmpty()) {
14         int shirt = Integer.parseInt(shirtStr);
15         if (shirt >= 1 && shirt <= 99 && !shirtTakenInClub(p.getClubID(), shirt, p))
16             p.setShirtNumber(shirt);
17         else System.out.println("This shirt number already exists in this club!");
18     }
19     dirty = true; System.out.println("Updated successfully.");
20 }
21 private boolean shirtTakenInClub(String clubId, int shirt, Player except) {
22     for (Player x : list)
23         if (x != except && x.getClubID().equals(clubId) && x.getShirtNumber() == shirt)
24             return true;
25     return false;
26 }

```

5.5 Đọc/ghi tệp văn bản (Functions 12, 13)

Listing 5.5. save() ghi text; loadStrict() validate từng dòng

```

1  public void save() {
2      try (PrintWriter pw = new PrintWriter(new FileWriter("clubs.txt"))) {
3          for (Club c : list)
4              pw.printf("%s, %s, %s, %.0f%n", c.getClubID(), c.getName(), c.getSponsor(),
                    c.getBudget());
5          dirty = false;
6          System.out.println("Club data saved to clubs.txt.");
7      } catch (IOException e) { System.out.println("Save failed: " + e.getMessage()); }

```



```

8 }
9 public boolean loadStrict(String path) {
10     List<Club> tmp = new ArrayList<>();
11     try (BufferedReader br = new BufferedReader(new FileReader(path))) {
12         String line;
13         while ((line = br.readLine()) != null) {
14             if (line.isBlank()) continue;
15             String[] p = line.split("\\s*,\\s*");
16             if (p.length != 4 || !Validatable.isValid(p[0], Validatable.CLUB_ID_REGEX)) return false;
17             double budget = Double.parseDouble(p[3]);
18             if (budget <= 0 || p[1].isEmpty() || p[2].isEmpty()) return false;
19             tmp.add(new Club(p[0], p[1], p[2], budget));
20         }
21     } catch (Exception e) { return false; } // any malformed line -> fail
22     list = tmp; return true;
23 }

```

Thông báo đúng chuẩn đề

Nạp lại (Function 13): xoá dữ liệu hiện tại, nạp club rồi player; nếu *bất kỳ* dòng nào sai → in "Load data failed!"; ngược lại "Load data successfully!".

5.6 Mẫu kết quả (Sample Output)

Listing 5.6. List + sort + search

```

Clubs:
Club ID | Club Name | Sponsor | Budget
CL-0001 | FC Barcelona | Nike | 820
CL-0002 | Real Madrid Galacticos | Adidas | 910

Players (by club name, then shirt):
P0003 | CL-0001 | Jordi Alba | Defender | 18
P0001 | CL-0001 | Lionel Messi | Forward | 10 <- shirt asc within club

Search "mess":
P0001 | CL-0001 | Lionel Messi | Forward | 10

```

5.7 Liệt kê & xoá cầu thủ, lọc theo vị trí, thoát

Listing 5.7. listAll(), remove(), listByPosition(), quit()

```

1 public void listAll() {
2     if (list.isEmpty()) { System.out.println("The club list is empty."); return; }
3     list.sort(null); // dùng Comparable<Club>: theo ten
4     System.out.printf("%-8s| %-24s| %-8s| %s\n", "Club ID", "Club Name", "Sponsor",
5         "Budget");
6     list.forEach(System.out::println);
7 }
8 public void remove() {
9     String id = Inputter.inputLoop("Player ID: ", Validatable.PLAYER_ID_REGEX, "Format must
10         be Pxxxx");
11     Player p = findById(id);
12     if (p == null) { System.out.println("This player does not exist!"); return; }
13     list.remove(p); dirty = true;
14     System.out.println("Player removed.");

```

```
13 }
14 public void listByPosition() {
15     String pos = Inputter.inputNonEmpty("Position: ");
16     boolean any = false;
17     for (Player p : list)
18         if (p.getPosition().equalsIgnoreCase(pos)) { System.out.println(p); any = true; }
19     if (!any) System.out.println("No players play in this position.");
20 }
21 public void quit(ClubsManager clubs, PlayersManager players) {
22     if (clubs.isDirty() || players.isDirty()) { clubs.save(); players.save(); } // save if
        changed
23     System.out.println("Goodbye!");
24 }
```

5.8 Phiên chạy mẫu đầy đủ

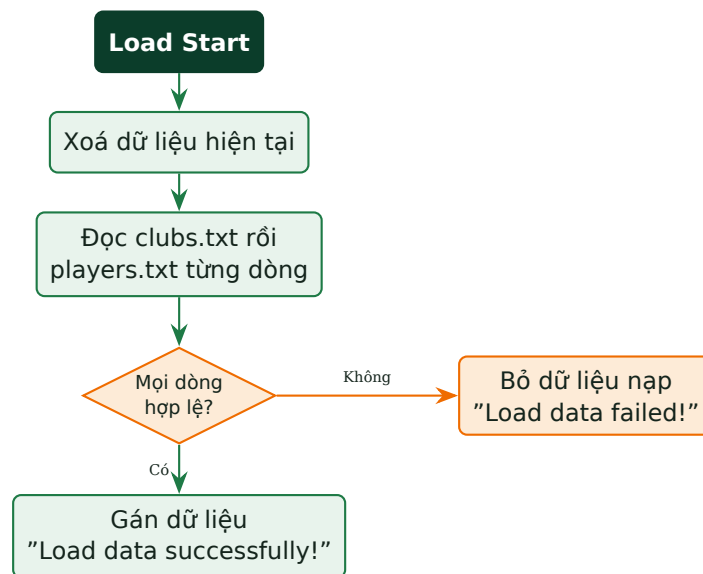
Listing 5.8. Một phiên thao tác minh họa

```
Choose 1-14: 8
Player ID: P9 -> Format must be Pxxxx
Player ID: P0016
-- Club list shown --
Club ID: CL-9999 -> This club does not exist!
Club ID: CL-0001
Name: Pedri
Position: midfielder
Shirt (1-99): 10 -> This shirt number already exists in this club!
Shirt (1-99): 16
Added successfully

Choose 1-14: 6
Players (by club name, then shirt):
P0003 | CL-0001 | Jordi Alba | Defender | 18
P0001 | CL-0001 | Lionel Messi | Forward | 10

Choose 1-14: 14
Club data saved to clubs.txt.
Player data saved to players.txt.
Goodbye!
```

5.9 Flowchart bổ sung: Load chặt (Function 13)



Hình 9: Flowchart bổ sung: Load chặt (Function 13)

6

Tiêu chí chấm (Rubric)

6.1 Cấu trúc tổng thể (thang 100, quy đổi 300 LOC)

Khối	Điểm	Ghi chú
Computational Thinking (CT)	20	Decomp 5 + Pattern 3 + Abstraction 6 + Algorithm 6
AI Reflection — CT	10	thuộc 30% AI Reflection
Áp dụng OOP	25	Class 6 + Encapsulation 4 + Ctor/toString>equals 5 + Inheritance 10
AI Reflection — OOP	10	thuộc 30%
Chức năng (CRUD + File I/O)	15	chi tiết bên dưới
AI Reflection — Functional	10	thuộc 30%
Code style & báo cáo	10	Naming 4 + Clean 4 + Report 2
Tổng	100	AI Reflection cộng dồn = 30%

6.2 Chi tiết chức năng (15đ) — bám đúng bảng trường

Chức năng	Điểm	Chức năng	Điểm
Display Club/Player (table)	1.5	Thêm Player (validation)	2
Thêm Club (validation)	2	Xoá Player	1.5
Tìm kiếm & cập nhật Club	1.5	Cập nhật Player	1.5
Lọc Club theo budget	1.5	Lọc Player theo Position	1.5
Lưu dữ liệu vào file	2		

(Tổng cộng 15đ; các mức điểm trên thang 100, quy đổi theo 300 LOC.)

6.3 Chi tiết CT (20đ) & OOP (25đ)

CT	Điểm	OOP	Điểm
Decomposition	5	Định nghĩa class (Club, Player...)	6
Pattern Recognition	3	Encapsulation	4
Abstraction	6	Constructor/toString>equals	5
Algorithm Design	6	Kế thừa/interface/polymorphism	10

6.4 Worked Example & lỗi mất điểm

Khối	Điểm	Nhận xét
CT (20)	17	Phân rã & abstraction tốt; thiếu flowchart cho Load.
OOP (25)	23	Comparator chaining + null-safe; thiếu equals cho Player.
Functional (15)	13	14 chức năng chạy; số áo trùng chưa chặn ở Update (đã sửa).
Code style (10)	8	Tên rõ; vài method dài.
AI Reflection (30)	25	20 core prompts; 2 hallucination; thiếu evidence 1 entry.
Tổng thô	86/100	Khá-tốt.

Lỗi thường gặp	Khối bị trừ
Không chặn số áo trùng trong cùng club.	Functional (Add/Update Player)
Sort không null-safe khi club name không tra được.	OOP / Functional
Load không validate chặt, vẫn nạp dòng hỏng.	Functional (Load)
Không có kế thừa/interface/polymorphism.	OOP (10đ)
AI log thiếu Human Delta.	AI Reflection (→ có thể = 0)

6.5 AI Reflection (30đ)

Áp dụng bộ tiêu chí ở §7.6. Ngưỡng đạt: 16–24 core prompts, ≥ 2 hallucination, Human Delta đủ 4 câu hỏi, evidence cho $\geq 70\%$ entries.

AI Audit Log & AI Reflection — Cách làm và cách chấm

7.1 AI Audit Log là gì và *không* là gì

KHÔNG phải là

- Nơi copy-paste *toàn bộ* lịch sử chat với ChatGPT/Gemini/Claude.
- Bộ sưu tập các prompt vặt ("khai báo mắng thế nào?").
- Bảng chứng "đã dùng AI" một cách hình thức.

LÀ

- Nhật ký các *quyết định quan trọng* khi làm việc với AI.
- Bảng chứng tư duy phản biện và tự chủ.
- Chứng minh sinh viên *hiểu* code và giải pháp, không phải copy.

Hình 10: AI Audit Log là gì và *không* là gì

7.2 Core Prompt vs. Auxiliary Prompt

Nguyên tắc vàng

Chỉ ghi những prompt mà **nếu không có nó, dự án sẽ KHÁC VỀ BẢN CHẤT** (kiến trúc, thiết kế, cách giải quyết) — không phải chỉ khác về syntax hay định dạng.

Loại	CORE — phải ghi	AUXILIARY — không ghi
Decision-Making	"So sánh dùng Map hay List để tra cứu theo ID?"	"Cách khai báo ArrayList trong Java?"
Problem-Solving	"Vì sao sort theo nhiều tiêu chí bị sai thứ tự?"	"Sửa lỗi thiếu dấu ; dòng 42."
Verification/Critique	"Regex <code>\d{6}</code> có bắt đúng 6 chữ số không, có thiếu <code>^\$</code> không?"	"Sinh getter/setter cho lớp Student."

Mẹo kiểm tra nhanh

Nếu *không có* prompt này mà dự án vẫn giống tới 90% → **LOẠI**. Nếu thiếu nó dự án khác về bản chất → **GHI**.

7.3 Yêu cầu số lượng (bài Assignment LAB211)

16-24

Core Prompts

≥ 2

Hallucination phát hiện

≥ 1

Core prompt / mỗi thành phần CT

Công thức gợi ý: **Số Core Prompts** = (số thành phần DTC) $\times$ (2-3 prompt/thành phần).
Mỗi thành phần CT (Decomposition, Pattern Recognition, Abstraction, Algorithms) phải có *ít nhất một* core prompt tương ứng.

7.4 Cấu trúc một entry: 7 phần

Mỗi entry trong Audit Log gồm 7 phần; phần thứ 7 (**Human Delta**) là quan trọng nhất.

1. Entry #	Số thứ tự (001, 002, ...)
2. Prompt Type	DECISION / PROBLEM-SOLVING / VERIFICATION
3. Stage/Component	Decomposition, Pattern Recognition, Abstraction, Algorithms
4. Problem/Context	Vấn đề đang gặp (1-2 câu ngắn gọn)
5. Prompt to AI	Câu lệnh gửi cho AI (<i>nguyên văn</i>)
6. AI Response (Summary)	Tóm tắt phản hồi của AI (2-4 câu)
7. Human Delta & Reflection	Trả lời đủ 4 câu hỏi bên dưới

Bốn câu hỏi bắt buộc trong Human Delta

Critical Thinking

AI đúng/sai ở đâu? Có hallucination không?

Contextualization

Bối cảnh thực tế mà AI không biết?

Creative Synthesis

Tôi đã thay đổi/kết hợp/tối ưu thế nào?

Decision Ownership

Quyết định cuối cùng và lý do?

Hình 11: Bốn câu hỏi bắt buộc trong Human Delta

7.5 Phát hiện Hallucination (bắt buộc)

Hallucination là khi AI đưa ra thông tin *sai* nhưng trình bày rất tự tin. Năm loại phổ biến:

Fabrication	AI bịa thông tin không tồn tại (API/hàm/tham số không có thật).
Oversimplification	AI bỏ qua edge case (mảng rỗng, trùng dữ liệu, null).
Logic Error	Kết luận sai về "best practice" ("luôn dùng X là tốt nhất").
Outdated Info	Dùng cú pháp/API cũ đã deprecated.
Context Misunderstanding	Không hiểu đúng ràng buộc nghiệp vụ của đề bài.

Quy trình xử lý khi nghi ngờ AI sai

(1) Nêu nghi ngờ cụ thể → **(2)** Kiểm chứng độc lập (đọc tài liệu chính thống, chạy thử, viết test) → **(3)** Ghi lại *cách phát hiện* và *hành động sửa chữa* (corrective action).

7.6 Tiêu chí chấm AI Reflection (30%)

Phần AI Reflection chiếm **30%** tổng điểm, gán vào ba mục tiêu (mỗi mục 10%): *AI Usage Documentation*, *Critical Thinking & Reflection on AI Output*, *Effective AI-assisted Learning Process*. Mỗi mục được chấm theo 4 mức.

Tiêu chí	Excellent (9–10)	Good (7–8)	Average (5–6)	Poor (0–4)
AI Usage Documentation	Ghi log đầy đủ; có prompt, kết quả AI, quyết định sử dụng/chỉnh sửa; rõ mục tiêu học tập.	Ghi phần lớn quá trình nhưng chưa đầy đủ, lập luận chưa thuyết phục/không nhất quán.	Có dùng AI nhưng log sơ sài, thiếu prompt hoặc thiếu giải thích (dùng mù quáng).	Không có AI Log hoặc chỉ copy output AI mà không giải thích.
Critical Thinking & Reflection	Phân tích đúng/sai dựa trên output AI; phát hiện lỗi; giải thích lý do chỉnh sửa; chứng minh hiểu mã nguồn.	Có phản biện kết quả AI nhưng chiều sâu phân tích chưa cao.	Chỉ mô tả AI hỗ trợ gì, ít phân tích đúng/sai.	Chấp nhận output AI thụ động; không chứng minh được hiểu biết.
Effective AI-assisted Learning	Dùng AI đúng vai trò: phân tích yêu cầu, thiết kế class, debug, refactor, giải thích thuật toán; có iterative prompting.	Dùng AI hợp lý nhưng chưa đa dạng hoặc thiếu chiều sâu.	Dùng AI chủ yếu để generate code cho xong việc.	Lạm dụng AI; không giải thích được sản phẩm hoặc quá trình học.

Pass/Fail trong Oral Vivas

Giảng viên sẽ hỏi vấn đáp 3–5 entries. Sinh viên **Pass** khi: giải thích được lý do quyết định cho $\geq 70\%$ entries được hỏi; có evidence cụ thể (code, metrics, so sánh); phát hiện ≥ 1 hallucination và giải thích cách fix; số entries nằm trong range. **Nếu FAIL ≥ 2 tiêu chí $\rightarrow$ AI Reflection = 0 điểm (mất 30%).**

7.7 Dấu hiệu nhận biết khi chấm

Dùng AI hiệu quả — sinh viên...	Dấu hiệu lạm dụng AI...
Biết chia nhỏ vấn đề; hỏi AI đúng trọng tâm.	Không giải thích được code; không hiểu logic.
Có iterative prompting; có debugging process.	AI log chỉ copy chat; không có reflection.
Có phản biện AI; có điều chỉnh AI output.	Không có decision making; code style không nhất quán.
Giải thích được code; liên hệ với nguyên lý OOP.	Không giải thích được tại sao chọn giải pháp đó.

7.8 Quy định cho sinh viên

Được phép (Allowed)	Không được phép (Not Allowed)
Giải thích kiến thức; học OOP; debug; refactor.	Nộp nguyên output AI mà không hiểu.
Gợi ý thuật toán; sinh code mẫu; giải thích lỗi.	Che giấu việc sử dụng AI.
Sinh test case.	Copy nguyên project từ AI; không giải thích được source code.

7.9 AI Literacy — viết prompt hiệu quả

Kỹ năng "Enhance AI Self Worth / AI Literacy" yêu cầu sinh viên hiểu nguyên tắc dùng GenAI, viết prompt rõ ràng, kiểm chứng thông tin và nắm rủi ro/đạo đức. Một prompt tốt thường có bốn thành phần:

Thành phần	Vai trò
Vai trò/Bối cảnh	"Tôi đang viết Java console theo OOP cho bài LAB211..."
Nhiệm vụ cụ thể	"...cần một Comparator sắp theo tên CLB rồi số áo."
Ràng buộc	"Không dùng thư viện ngoài; xử lý null an toàn."
Định dạng đầu ra	"Trả về đoạn code + giải thích từng bước."

Ví dụ iterative prompting (nhiều vòng)

Chuỗi prompt cải thiện dần

Vòng 1: "How to sort a list by two fields in Java?" → AI cho `Comparator.comparing().thenComparing()`.

Vòng 2 (thu hẹp): "What if some club names are null — will it throw NPE?" → AI gợi ý `Comparator.nullsLast()`.

Vòng 3 (kiểm chứng): "Show me a JUnit-style test proving order is correct for 3 sample items." → tôi chạy thử, xác nhận, và *tự quyết định* dùng phương án có null-safety.

7.10 Phương pháp Oral Vivas (vấn đáp)

Giảng viên đánh dấu 3-5 entries trong log và hỏi sâu khoảng 2-3 phút/entry. Bộ câu hỏi mẫu:

Loại prompt	Câu hỏi vấn đáp tiêu biểu
Decision-Making	"Tại sao em chọn approach B mà không phải A? Nhược điểm của B là gì? Em đã thử cả hai chưa, khác biệt bao nhiêu?"
Problem-Solving	"Lỗi gốc nằm ở đâu? Em đã khoanh vùng thế nào? Cách fix có tạo lỗi mới không?"
Verification	"Em phát hiện AI sai ở entry nào? Kiểm chứng bằng cách gì? Sau đó em làm gì?"

Dấu hiệu "log giả" (viết sau khi làm xong)

Sinh viên KHÔNG trả lời được vì sao chọn approach A thay vì B; KHÔNG nhớ AI đã trả lời gì; KHÔNG có evidence (ảnh chụp, diff code, số đo). Khi đó dù log "đẹp" vẫn bị đánh giá thấp.

7.11 Bảng phân bố entries theo thành phần CT

Khi nộp, sinh viên kèm bảng tóm tắt để giảng viên kiểm tra nhanh độ phủ:

Thành phần CT	Số entry	Yêu cầu	Loại prompt tiêu biểu
Decomposition	≥ 1	≥ 1	chia module, chọn cấu trúc dữ liệu
Pattern Recognition	≥ 1	≥ 1	regex, CRUD template
Abstraction	≥ 1	≥ 1	chọn thuộc tính/lớp, interface
Algorithms	≥ 1	≥ 1	sort, search, thống kê, file I/O
Tổng core prompts	16-24 (kèm ≥ 2 hallucination đã phát hiện)		

Ví dụ AI Audit Log cho Football

8.1 Entry ĐẠT — Verification (regex CL-xxxx)

[✓] Entry #003 — VERIFICATION — Pattern Recognition

Prompt to AI: "Java regex for club ID like CL-0001 (CL dash four digits)."

AI Response: `CL-\d{4}`.

Human Delta: *Critical Thinking:* thiếu neo `^...$` và trong String Java phải là `"^CL-\d{4}$"`. *Decision Ownership:* đưa vào `Validatable`; viết test cho CL-0001 (đạt), CL-1 (loại), `cl-0001` (loại do phân biệt hoa thường ở tiền tố).

8.2 Entry ĐẠT — Decision (Comparator chaining)

[✓] Entry #009 — DECISION-MAKING — Algorithms

Prompt to AI: "Sort players by club name then by shirt number in Java."

AI Response: gợi ý `Comparator.comparing(...).thenComparing(...)`.

Human Delta: *Critical Thinking:* club name lấy qua `clubs.getName(clubId)` có thể null → NPE. *Creative Synthesis:* thêm `nullsLast(CASE_INSENSITIVE_ORDER)` và `thenComparingInt`. *Decision Ownership:* chốt giải pháp null-safe; kiểm chứng với một player thuộc club không tồn tại.

8.3 Entry phát hiện HALLUCINATION (số áo & vị trí)

Entry #012 — Hallucination: Oversimplification / Context Misunderstanding

AI's Claim: "Chỉ cần kiểm tra số áo 1-99 là đủ."

Reality Check: Đề yêu cầu số áo *duy nhất trong cùng club* — AI bỏ qua ràng buộc nghiệp vụ.

How Detected: Viết test thêm hai cầu thủ cùng club, cùng số áo → chương trình AI gợi ý chấp nhận (sai).

Corrective Action: Thêm `shirtTakenInClub(clubId, shirt)`; in đúng thông báo của đề.

8.4 Entry KÉM (để đối chiếu)

[✗] Entry #006 — không đạt

Human Delta: "AI viết hộ hàm sort, chạy ok."

Vì sao kém: không nêu rủi ro null, không giải thích quyết định, không evidence → **LOẠI**.

8.5 Ngân hàng câu hỏi vấn đáp (Football)

Chủ đề	Câu hỏi mẫu
Quan hệ 1-nhiều	"Vì sao thêm Player phải kiểm tra Club tồn tại? Em chặn ở đâu trong code?"
Số áo duy nhất	"Làm sao đảm bảo số áo duy nhất <i>trong cùng club</i> mà không phải toàn hệ thống?"
Comparator	"Giải thích thenComparingInt. Nếu club name null thì sao?"
Load chặt	"Khi một dòng hỏng, vì sao phải fail toàn bộ thay vì bỏ qua dòng đó?"
Polymorphism	"Comparable vs Comparator khác nhau thế nào trong bài này?"

8.6 Chỉ mục Audit Log mẫu (20 core prompts)

Entry	Type	Component	Tóm tắt
#001	Decision	Decomposition	Tách Club/Player + manager + tools.
#002	Decision	Abstraction	Person abstract làm gốc cho Player.
#003	Verification	Pattern Recognition	Regex CL-xxxx, thêm neo & escape.
#004	Verification	Pattern Recognition	Regex Pxxxx cho player.
#005	Decision	Abstraction	Position dùng Set lowercase (case-insensitive).
#006	Problem-Solving	Algorithms	Lọc club theo budget.
#007	Problem-Solving	Pattern Recognition	Partial name search, lowercase.
#008	Decision	Algorithms	Cấu trúc lưu Player để tra số áo theo club.
#009	Decision	Algorithms	Comparator chaining (club name, shirt).
#010	Problem-Solving	Algorithms	shirtTakenInClub khi Add/Update.
#011	Problem-Solving	Abstraction	Update giữ field nếu Enter rỗng.
#012	Verification	Algorithms	Hallucination: bỏ qua ràng buộc số áo trong club.
#013	Verification	Algorithms	Hallucination: split(",") không trim khoảng trắng.
#014	Problem-Solving	Algorithms	Load chặt: fail nếu có dòng sai.
#015	Decision	Algorithms	Lưu text vs object (chọn text theo đề).
#016	Problem-Solving	Pattern Recognition	isPosition không phân biệt hoa thường.
#017	Problem-Solving	Algorithms	Budget phải dương khi nạp & nhập.
#018	Decision	Decomposition	Tách ValidationUtils impl Validatable.
#019	Problem-Solving	Algorithms	Reload xoá dữ liệu cũ trước khi nạp.
#020	Verification	Decomposition	Rà soát đủ 14 chức năng so với đề.

Phụ lục — Checklist tự chấm

#	Tiêu chí	Đạt?
1	14 chức năng chạy đúng, đúng thông báo của đề	☐
2	Số áo duy nhất trong cùng club (Add & Update)	☐
3	Sort null-safe (Function 6)	☐
4	Load fail khi có dòng hỏng	☐
5	Có kế thừa/interface/polymorphism	☐
6	16–24 core prompts, ≥ 2 hallucination	☐
7	Human Delta đủ 4 câu hỏi mỗi entry	☐

Lời kết

Football mở rộng Mountain ở *quan hệ 1-nhiều*, sắp xếp đa tiêu chí và ràng buộc theo nhóm (số áo trong club). Nắm vững Comparator, null-safety và validate chặt khi nạp tệp là chìa khoá đạt điểm OOP & Functional.